# Guida tecnica per modificare il simulatore Cherenkov v78

Guida per chi deve orientarsi nel codice, cambiare una via di simulazione o aggiungere una regressione senza confondere l'orchestrazione della GUI con il modello fisico.

> Conferma della versione. v78 è confermata dai file caricati in questo turno: il README riporta la versione applicativa 0.46.0 e il build ID build-20260929-v78-gamma-charged-boundaries; lo stesso identificativo compare nel modulo GUI e nel riepilogo di implementazione. Questa guida usa quei file come fonte e non si basa su contesto precedente della knowledge base.

## Ambito e livello di certezza

La consegna contiene ora i quattro moduli che definiscono il flusso completo: quartz_cherenkov_gui.py orchestra input e run; quartz_cherenkov_sim.py implementa emissione e trasporto ottico; quartz_geometry.py fornisce i modelli canonici di assemblaggio, trasformazione e serializzazione; gamma_transport.py implementa il trasporto Gamma di prima fase. quartz_geometry_editor.py contiene anche un editor a sezioni autonomo e funzioni di supporto per gli editor canonici.

Le dichiarazioni sul comportamento ottico e geometrico qui sotto sono basate sul codice ricevuto, non dedotte dalle sole schermate o dai README. Il simulatore offre modelli Monte Carlo eseguibili; questo non equivale a una validazione sperimentale delle formule o dei parametri. La guida separa quindi ciò che il codice calcola da ciò che i file dichiarano esplicitamente come limite.

I duplicati quartz_cherenkov_gui_1.py, gamma_transport_1.py, check_environment_1.py e test_gamma_transport_1.py hanno gli stessi hash SHA-256 dei rispettivi file senza suffisso; il loro contenuto è identico. Per l'analisi sono state usate le copie originali. I tre moduli aggiunti quartz_cherenkov_sim.py, quartz_geometry.py e quartz_geometry_editor.py sono file distinti senza variante _1 ricevuta.

Restano assenti alcuni artefatti di distribuzione e input: in particolare gamma_source_profile_collimated_v1.json, i launcher Windows originali e quartz_cherenkov_simulator_v78.zip. Il codice del lettore Gamma definisce lo schema del profilo, ma senza il JSON non si possono confermare i bin e la direzione del profilo distribuito. Il checkout allegato è sufficiente per ispezionare i moduli core, non coincide con l'intero pacchetto di rilascio atteso dal checker.

## Mappa dei file

| File | Ruolo verificabile |
|---|---|
| quartz_cherenkov_gui.py | GUI Tkinter: input, convalida, snapshot immutabile, conversione unità, modalità, worker, persistenza e viste. Contiene anche l'adattamento dei modelli geometrici al run. |
| quartz_cherenkov_sim.py | Modelli ottici Cherenkov e Scintillator, solidi e composti, generazione Monte Carlo, trasporto dei fotoni, record, riepiloghi, output ROOT/JSON, grafici e CLI. |
| quartz_geometry.py | Materiali e sezioni, GeometryModel storico, AssemblyGeometryModel canonico, validazione terminali/sensori, conversione ai solidi ottici, trasformazioni rigide e serializzazione JSON. |
| quartz_geometry_editor.py | Helpers per assemblaggi canonici e classe Tk GeometryEditorPanel del modello storico a sezioni; righe editabili, round-trip, preview 2D/3D e salvataggio PNG. |
| gamma_transport.py | Coefficienti efficaci per Gamma, sorgente e volumi, interazioni, tracce cariche rettilinee, depositi, luce locale, riepiloghi, JSON e CLI. |
| test_gamma_transport.py | Test mirati sul range noto, chiusura energetica, genealogia delle tracce e attraversamento di volumi adiacenti. |

| File | Ruolo verificabile |
|---|---|
| check_environment.py | Checker headless: compilazione e controlli/regressioni su GUI, modelli ottici, geometrie, sensori, scene, risultati, Gamma e viewer; verifica condizionale di archivio e launcher. |
| README_v78.md | Identità di rilascio, cambiamenti v78, limiti e comandi di verifica. |
| README_gamma_transport_stage.md | Scopo e limiti dello stadio Gamma di prima fase. |
| implementation_summary_v78.txt | Provenienza e riepilogo delle modifiche che identificano v78. |
| gamma_range_model_diagnostic.md | Formula legacy del range elettronico, discrepanza nota e riferimento necessario prima di cambiarla. |

Il profilo Gamma predefinito, l'archivio ZIP, i file storici richiesti dal controllo di pacchetto e i launcher Windows non sono inclusi nella cartella corrente. Questo limita la verifica del contenuto di distribuzione e del profilo reale, non l'ispezione dei moduli ottici e geometrici ora presenti.

## Architettura: dall'interfaccia alla simulazione

La GUI non esegue il trasporto nel thread Tk. Converte i campi testuali in un oggetto tipizzato, valida la selezione, cattura un'istantanea indipendente dello stato, quindi avvia un worker. Il worker ricostruisce i propri oggetti e restituisce aggiornamenti tramite una coda.

> Run ottico: campi GUI e modelli → validate_settings → _make_run_snapshot → QuartzCherenkovGUI._worker_run → run_simulation_to_directory → quartz_cherenkov_sim.simulate_events → summary, JSON, ROOT opzionale e PNG.
>
> Run Gamma: campi GUI e modelli → validate_settings → _make_run_snapshot → QuartzCherenkovGUI._worker_run → run_simulation_to_directory → run_gamma_transport_to_directory → gamma_transport.simulate → gamma_results.json, summary.json e PNG.
>
> Risultati: artefatti e riepiloghi → viste della GUI.

I nomi indicano funzioni o metodi del sorgente GUI e del modulo ottico, ora entrambi inclusi. La chiamata finale è tracciabile fino a simulate_events e simulate in quartz_cherenkov_sim.py.

### Punti di ingresso importanti

| Punto di ingresso | Responsabilità |
|---|---|
| QuartzCherenkovGUI.main | Verifica Tkinter, costruisce la GUI e avvia il main loop. |
| QuartzCherenkovGUI._run | Rifiuta run concorrenti, controlla la geometria, chiama validate_settings e validate_canonical_run_assemblies, crea la directory e lo snapshot e avvia il thread. |
| QuartzCherenkovGUI._worker_run | Ricostruisce RunSettings, i modelli e la geometria dal RunSnapshot; invoca run_simulation_to_directory; invia esito o errore in coda. Il worker non legge StringVar o widget Tk. |
| validate_settings | Converte il dizionario di valori della GUI in RunSettings, controlla intervalli e modalità e, in modalità Gamma, apre il profilo attraverso GammaSource.from_profile. |
| run_simulation_to_directory | Separa il ramo Gamma dagli altri run; nel ramo ottico crea Particle, OpticalConfig e configurazione Scintillator, quindi invoca simulator.simulate_events. Cura log, riepiloghi e artefatti. |
| run_gamma_transport_to_directory | Costruisce il geometry snapshot globale, legge il profilo, crea i TransportVolume dalle geometrie congelate, lancia gamma_transport.simulate e salva i prodotti Gamma. |

L’oggetto RunSnapshot è una struttura di dati congelata composta da impostazioni, modelli di assemblaggio selezionati e geometria di trasporto serializzata. _make_run_snapshot copia i modelli prima di serializzarli e rende immutabili i contenitori. Questo è un confine di concorrenza, non solo un modo di passare parametri: una modifica futura non dovrebbe far leggere al worker i widget o i modelli ancora modificabili dall’editor.

### Due percorsi, due scopi

Le modalità ottiche GUI sono Cherenkov only, Scintillator only e All; il valore All viene normalizzato a combined. Gamma transport è una modalità autonoma, non un’opzione che aggiunge interazioni Gamma al run ottico.

Il worker ricostruisce geometrie e impostazioni dal RunSnapshot. Per il run Cherenkov/Scintillator/All crea Particle e OpticalConfig; se partecipa lo scintillatore, costruisce anche ScintillatorAssembly e ScintillatorConfig. La funzione GUI run_simulation_to_directory invoca simulator.simulate_events. Questa crea generatori casuali figli dal seed del run e chiama simulate una volta per evento.

In modalità cherenkov, simulate traccia la particella nella geometria attiva, integra lo spettro Frank–Tamm e trasporta i campioni ottici. In modalità scintillator salta emissione e trasporto Cherenkov e usa il volume attivo scintillante. In combined esegue entrambi i contributi nello stesso risultato; i record dello scintillatore sono conservati nel campo dedicato ScintillatorResult. La costruzione del percorso e la semantica delle modalità sono quindi nel codice, non soltanto nella GUI.

Il percorso Gamma della GUI chiama run_gamma_transport_to_directory, che adatta le geometrie congelate a TransportVolume e invoca gamma_transport.simulate. Il ramo salva gamma_results.json, gamma_transport.png e summary.json; non produce ROOT o readout_arrivals.json e non trasforma LightProduction in fotoni ottici da trasportare. I due modelli condividono una formula Frank–Tamm per la stima locale, ma hanno pipeline e risultati separati.

## Algoritmi ottici: generazione e trasporto dei fotoni

Il punto d’ingresso per un singolo evento è simulate; per una sequenza di eventi è simulate_events. Le principali strutture pubbliche sono Particle, OpticalConfig, ScintillatorAssembly, ScintillatorConfig, PhotonRecord e SimulationResult. La GUI è un adattatore: converte le unità e i profili di run, mentre queste funzioni determinano campionamento, traiettorie e conteggi.

### Emissione Cherenkov

1. particle_beta calcola beta dall’energia cinetica e dalla massa relativistica. direction_from_angles e Particle.direction definiscono il vettore unitario: l’inclinazione è misurata rispetto a +Z; l’azimut zero punta verso +X. La GUI imposta l’azimut a zero, perciò la traiettoria GUI inclinata giace nel piano X-Z.
2. La geometria restituisce il tratto della particella nel solido. Può essere una geometria rettangolare compatibile, un SolidGeometry o un CompoundGeometry; quest’ultimo individua anche il materiale locale lungo il tratto. La perdita di energia è opzionale e, se fornita come dEdx_J_per_m, è costante lungo il cammino.
3. frank_tamm_spectrum calcola $dN/(dx\,d\lambda)$. Per ogni lunghezza d’onda il contributo è nullo quando $\beta n \le 1$; altrimenti è proporzionale a $1 - 1/(\beta^2 n^2)$ e a $1/\lambda^2$. Il simulatore integra numericamente lo spettro e il tratto per ottenere il numero atteso di fotoni. Il range predefinito di OpticalConfig è 300–650 nm; il profilo usa una griglia in lunghezza d’onda e 256 intervalli di posizione lungo la traccia.
4. La scelta di quanti fotoni trasportare dipende da sampling_mode. fixed trasporta photon_count campioni di Monte Carlo e lo usa come taglia del campione, non come resa fisica. poisson estrae il numero di fotoni con media pari all’aspettativa integrata. L’aspettativa resta nel riepilogo in entrambi i casi.
5. Nel modo fisico, la posizione di emissione è scelta con probabilità proporzionale alla resa integrata locale; la lunghezza d’onda è campionata dalla CDF dello spettro locale. L’azimut intorno all’asse della particella è uniforme e l’angolo Cherenkov soddisfa $\cos\theta = 1/(\beta n)$. Il tempo iniziale del fotone è il tempo della particella per arrivare a quel punto.

L’indice del quarzo/fused silica usa la relazione Sellmeier codificata in fused_silica_index. SolidGeometry restituisce n=1 per air, n=1.58 per scintillator, usa Sellmeier per fused silica e per gli altri materiali senza indice esplicito, oppure un indice costante se refractive_index_value è presente. La relazione di indice non è automaticamente una tabella ottica specifica del materiale per ogni etichetta: in particolare non si deve dedurre una dispersione del PMMA dal solo nome light_guide. La tabella materiale dello stadio Gamma è un modello distinto.

## Cammino del fotone e interazioni ai confini

_transport_photon porta il fotone dalla posizione di emissione fino a un esito terminale. A ogni tratto calcola la prossima intersezione geometrica; assorbimento e scattering Rayleigh opzionali competono con quella distanza attraverso campioni esponenziali. Il tempo ottico accumulato è distanza × indice / c.

- L’assorbimento termina il fotone. Lo scattering Rayleigh lo ridistribuisce in una direzione isotropa; il contatore scattering aumenta e il trasporto continua dal punto spostato numericamente oltre il punto d’interazione.
- Alle interfacce interne di CompoundGeometry, il codice confronta gli indici dei due materiali e usa Fresnel non polarizzato; i fotoni trasmessi seguono la rifrazione di Snell, quelli riflessi seguono la riflessione speculare. La riflessione totale interna è registrata separatamente.
- Sulle superfici esterne, il codice supporta roughness angolare, riflettività Mylar nei solidi composti avvolti e terminali open, mirror o detector. Open termina come fuga; mirror applica una probabilità di riflessione configurata; detector applica il passaggio Fresnel verso il mezzo di accoppiamento.
- Il detector ha una finestra attiva rettangolare, circolare o ellittica sulla sezione Y-Z, con eventuale offset e rotazione per OpticalConfig. La validazione controlla che l’apertura entri nel portale. Dopo la trasmissione, il record distingue area attiva, accettanza angolare, efficienza di raccolta e QE; il fotone è detected solo se supera tutti i passaggi.

PhotonRecord distingue detector_transmitted, detector_valid_sensor_hit, readout_arrival_time_s e detected. L’arrivo di readout è il passaggio fisico nella finestra finita prima dei tagli angolari, della raccolta e della QE; l’arrivo detected è sottoposto anche a tali efficienze. I record contengono outcome e motivo finale, conteggi di riflessioni/scattering e punti del percorso. Il trasporto si ferma anche a max_bounces; lo stop cooperativo viene controllato nel ciclo dei fotoni e può restituire un evento parziale marcato stopped.

La funzione readout_arrival_analysis separa i record per proprietario (Cherenkov o Scintillator), costruisce analisi selezionate per evento e aggregate e può ricostruire le distribuzioni dai record persistiti. write_readout_arrivals scrive lo schema readout-arrivals-v1 in nanosecondi tramite file temporaneo e sostituzione atomica.

## Scintillazione ottica

simulate_scintillator deriva i segmenti della particella che attraversano le sezioni attive. L’energia depositata è, in ordine, l’override deposited_energy_MeV, dEdx_MeV_per_m × lunghezza attiva, oppure il dEdx_J_per_m della particella convertito in MeV; senza questi input è zero. L’aspettativa è energia depositata × photon_yield_per_MeV.

In physical/poisson il numero campionato è Poisson con la media attesa; in fixed è photon_count. La posizione viene campionata uniformemente sulla lunghezza delle sezioni attive attraversate, le direzioni sono isotrope e le lunghezze d’onda uniformi nell’intervallo impostato (default 350–550 nm). Il ritardo è esponenziale con media decay_time_s; zero significa emissione istantanea. Ogni fotone entra nel medesimo trasporto geometrico Fresnel/assorbimento/terminali usato dal simulatore ottico, con i limiti di rimbalzo, readout e Mylar configurati.

Se la traccia non attraversa il volume ma è stato impostato un deposito esplicito, il modo diagnostico può collocare la sorgente nel centro della sezione attiva invece di inventare un cammino carico: il track length resta zero. La geometria scintillante ha un suo frame e i record riportano path_coordinate_frame, ingresso/uscita dalla guida e contatori d’interfaccia. La funzione enforce_scintillator_readout_invariant controlla prima del riepilogo che l’estremo di readout corrisponda al sensore configurato.

### Riepiloghi e output del simulatore ottico

SimulationResult.summary e aggregate_summary separano campioni richiesti, generati, trasportati, attraversamenti del portale, arrivi, hit validi e fotoni rilevati; riportano anche medie, motivi di uscita, indici di soglia, lunghezze di traccia e, se presente, il riepilogo Scintillator. Tra i campi principali figurano detector_mode, requested_photons, generated_photons, physical_expected_photons, transported_photons, detector_face_crossings, valid_sensor_hits, readout_arrivals, detected_photons, outcomes e outcome_reasons; il summary aggregato conserva anche i riepiloghi per evento. I plot_* producono visualizzazioni per evento, conteggi cumulativi, fotoni per evento, tempi, lunghezze d'onda ed esiti per faccia. write_root offre output ROOT quando le dipendenze opzionali sono disponibili; il JSON readout è un formato distinto per l'analisi temporale. Gli errori nei prodotti opzionali sono gestiti separatamente dal calcolo del riepilogo nella GUI.

## Modello geometrico, assemblaggi ed editor

Nel progetto convivono tre interfacce geometriche con scopi diversi. La GUI principale usa AssemblyGeometryModel in quartz_geometry.py, che conserva in millimetri le sezioni attive, le guide, i terminali e i sensori. Il simulatore ottico consuma SolidGeometry o CompoundGeometry in metri. quartz_geometry_editor.py fornisce inoltre un editor a sezioni storico basato su GeometryModel; non è il tipo passato direttamente al trasporto.

### Solidi usati dal trasporto ottico

- BarGeometry conserva il caso compatibile rettangolare centrato all'origine: lunghezza X 100 mm, larghezza Y 20 mm e spessore Z 10 mm di default.
- SolidGeometry è un solido convesso con centro in metri. I tipi sono box, tapered_rectangular_prism, cylinder, elliptical_cylinder e truncated_cone. Il prisma rastremato varia larghezza Y e spessore Z lungo X; cilindro, cilindro ellittico e cono tronco usano l'asse X, Y o Z selezionato.
- SolidGeometry.validate controlla forma, materiale, asse, centro finito e dimensioni positive. contains, ray_intersection e ray_interval forniscono containment, normale e intersezioni di raggio; surface_rings fornisce profili per la visualizzazione.
- CompoundGeometry dispone più sezioni in sequenza lungo X da −X a +X. Il modello ordinario richiede contatto tra sezioni e la stessa sezione Y/Z sui due lati dell'interfaccia; respinge vuoti, sovrapposizioni e sporgenze ambigue. Le sezioni curve in una composizione devono essere allineate su X. L'adattatore dell'assemblaggio può abilitare esplicitamente l'interfaccia con ingombri diversi per i solidi scintillatore/guida.

CompoundGeometry conserva l'indice di sezione locale mentre il raggio avanza: next_boundary distingue facce esterne e interfacce, refractive_index_at e material_at leggono la sezione corrente. Le facce terminali detector del modello sono i portali piani ±X; una forma o un asse senza portale compatibile non diventa automaticamente un sensore. I metodi detector_portal_dimensions e active_area_contains espongono rispettivamente le dimensioni e il test della finestra.

### Assemblaggio canonico e scena globale

AssemblySection espone dimensioni e centri in mm, nome, materiale, forma e asse; to_simulation_geometry converte mm in metri e crea SolidGeometry. AssemblyGeometryModel contiene sezioni attive in ordine semantico, sezioni guida opzionali, abilitazione e riflettività Mylar, punti d'impatto e stato dei terminali ±X. valid_guide_sections ignora righe guida incomplete durante l'editing, mentre validate/to_simulation_geometry rifiutano le guide abilitate ma non valide prima del trasporto.

Il terminale è open, mirror o Detector/Photosensor coupling. readout_status richiede esattamente un terminale di readout e, se esiste una guida valida, richiede che il readout sia al suo terminale di uscita. validate_complete_assembly confronta la finestra del fotosensore con la faccia fisica a cui è accoppiata. to_simulation_geometry ordina le sezioni nella direzione fisica −X→+X, produce un solido o una CompoundGeometry e allega terminal_interfaces e sensor_components. Questi ultimi metadati rendono coerenti geometria trasportata, aperture e renderer; lo stato terminale è la scelta che guida l'interazione del fotone.

build_complete_assembly_scene e complete_scene costruiscono componenti fisici e overlay di scena: corpo/finestra del sensore, facce di interfaccia e superfici Mylar. Gli overlay Mylar sono geometria di visualizzazione derivata dalle facce; non sono ulteriori volumi di trasporto. I serializer assembly_geometry_to_dict/from_dict e complete_detector_assembly_geometry_to_dict/from_dict salvano e ripristinano profili, terminali e impostazioni sensore, con controlli di forma/versione. I nomi terminali storici sono normalizzati da terminal_surfaces_from_payload e normalize_terminal_surface.

RigidAssemblyTransform applica una rotazione destrorsa comune: sottrae l'origine di rotazione, applica la matrice sull'asse X/Y/Z, aggiunge l'origine e infine la traslazione. I punti trasformati e i vettori di direzione seguono la stessa convenzione usata dai renderer e dal viewer Gamma. Nel run Gamma, la GUI converte anche le sezioni a TransportVolume tramite trasformazione rigida; lo snapshot di scena e i solidi interrogati devono derivare dallo stesso RunSnapshot.

### Il modulo quartz_geometry_editor.py

Il modulo esporta helpers canonici come canonical_assembly_model_from_sections, canonical_complete_assembly_scene, canonical_sensor_components, canonical_mylar_overlay_components, save/load_complete_detector_assembly e reorder_canonical_sections. Servono a editor, renderer e chiamanti headless per usare le stesse strutture e serializzazioni. Quando Tk/ttk è disponibile, GeometryEditorPanel è una classe Tk del modello storico GeometryModel: aggiunge/rimuove sezioni, aggiorna proprietà, converte con geometry_to_rows/ geometry_from_rows, carica/salva JSON e genera preview PNG/3D. La preview mostra proiezioni XZ, XY e YZ più la vista 3D.

GeometryModel è una sequenza di GeometrySection con lunghezze in X e larghezze/spessori iniziali/finali. I bordi X derivano dalle lunghezze di sezione riscalate alla lunghezza totale; dimensioni e materiale locali si interrogano con dimensions_at_x e material_at_x. ActiveDetectorArea può seguire l'intera faccia o definire un'apertura rettangolare, circolare o ellittica. MaterialSpec calcola fused silica con Sellmeier e, per il modello storico, indici costanti per aria/vuoto, acqua, PMMA/acrilico o preset custom. GeometrySection.dimensions_at interpola linearmente larghezza e spessore; i nomi storici di forma del modello non sono una conversione alle primitive 3D del simulatore. Questi tipi appartengono al percorso editoriale storico: per il trasporto del run GUI la conversione canonica è AssemblySection.to_simulation_geometry.

## Interfacce di configurazione e stato

### Impostazioni comuni della GUI

default_values fornisce valori testuali per le Tk StringVar; validate_settings ne effettua il parsing e costruisce la dataclass RunSettings. Le impostazioni sono molte e conviene raggrupparle per responsabilità:

- Run e riproducibilità: directory output, eventi, seed, modalità di campionamento, numero di fotoni richiesti per evento e massimo di rimbalzi.
- Particella incidente nel run ottico: tipo, energia cinetica, massa, coordinate d'ingresso, inclinazione e azimut conservato per compatibilità dei profili.
- Sensore Cherenkov e uscita opposta: indice, accettanza, QE, efficienza di raccolta, forma/dimensioni/ offset/rotazione dell'area attiva, modalità open/mirror/detector e riflettività.
- Assemblaggi Cherenkov e Scintillator: sezioni solide, materiale, dimensioni, guide, terminali, rivestimento Mylar e trasformazioni globali.
- Scintillazione ottica: conteggio o resa, energia depositata o dE/dx, tempo di decadimento, banda di lunghezza d'onda, assorbimento e parametri di readout.
- Gamma: percorso al profilo, pesi di linee esplicite, limite di interazioni e massimo di campioni di luce.

I valori predefiniti visibili nel sorgente includono 10 eventi, 100 fotoni/evento, energia cinetica ottica di 2.0 MeV, particella electron, seed 12345, modalità Cherenkov only e massimo 1000 rimbalzi. Il run Gamma è opt-in; i pesi di linea partono da zero, il limite è 1000 interazioni e il massimo di campioni di luce è 10000. Questi sono default GUI, non valori universali del modello.

validate_settings rifiuta, tra le altre cose, eventi minori di uno, conteggi negativi, energia cinetica negativa, massa non positiva, inclinazioni fuori da −89°…+89°, QE/efficienze/riflettività fuori da 0…1, seed o numeri non interpretabili e limiti Gamma non validi. Le convalide degli assemblaggi dipendono dalla modalità selezionata: Gamma e All richiedono entrambi i modelli completi Cherenkov e Scintillator nel RunSnapshot.

La GUI mantiene compatibilità con vecchi dizionari che hanno auto/automatic come modalità: se manca una scelta canonica, il vecchio campo scintillator_enabled viene usato per migrare una volta a combined o cherenkov. Per nuovi stati, detector_run_mode è l'autorità.

### Interfacce dirette dei modelli

Per modifiche al nucleo ottico, Particle riceve massa in kg, energia cinetica in joule, impatto X/Y in metri, angoli in radianti, carica in unità elementari e un eventuale dEdx in joule/metro. OpticalConfig raccoglie banda, numero e modalità di campionamento, lunghezze di assorbimento/scattering, rugosità, rimbalzi, risposta del sensore e finestre attive per entrambi i terminali. Questi oggetti sono dataclass: la GUI li costruisce nei propri adattatori, ma la CLI/self-test e i chiamanti Python possono costruirli direttamente.

simulate accetta un singolo evento, seed/event_id, stop_event e callback facoltative; simulate_events accetta il numero di eventi e callback di progresso/stato. aggregate_summary aggrega SimulationResult. Le callback non cambiano il modello fisico; il flag di stop cooperativo può restituire risultati parziali e impostare stopped. ScintillatorConfig aggiunge resa, energia depositata o dE/dx, ritardo medio, banda, readout e assorbimento. ScintillatorAssembly separa la geometria di trasporto, le sezioni attive/guida, i terminali, l'involucro Mylar e il frame dei record.

Per la geometria canonica della GUI, i punti di partenza sono AssemblySection e AssemblyGeometryModel; per i test geometrici del simulatore sono SolidGeometry e CompoundGeometry. Evitare modifiche a un solo tipo: seguire sempre la conversione e i renderer che consumano lo stesso stato.

### Configurazione Gamma

GammaSource.from_profile richiede schema gamma-source-profile-v1. Legge i bin come coppie di energia centrale in keV e peso relativo positivo; scarta righe con energia o peso non positivo e richiede almeno un bin valido. La direzione proviene da source_geometry.direction_global e deve essere finita e non nulla; il trasporto la normalizza quando la usa. Il profilo presente nella consegna non è disponibile, quindi non sono verificabili i suoi contenuti effettivi.

Le linee presenti nel profilo sono inattive se non viene passato un peso utente positivo. La GUI espone pesi per 511, 835, 1173, 1275, 1332 e 1461 keV. La selezione di una componente confronta il peso complessivo delle linee attive con il peso del continuo; all'interno della componente scelta si estrae un bin o una linea secondo i pesi relativi. I bin forniscono energie discrete ai loro centri: in questo codice non viene campionata una posizione casuale all'interno dell'ampiezza del bin.

### Cambiare un campo senza perdere coerenza

Quando si aggiunge un nuovo parametro di run, controllare almeno default_values, RunSettings e validate_settings. Verificare poi come viene serializzato in RunSnapshot, ricostruito dal worker e incluso nei riepiloghi. Per le modalità ottiche, il parametro può dover essere convertito in _optical_for_settings o nella configurazione Scintillator. Per Gamma, controllare inoltre il punto d'ingresso run_gamma_transport_to_directory e l'argomento effettivamente passato a gamma_transport.simulate. Evitare di aggiungere una seconda autorità a un campo che la GUI descrive come unico controllo: il limite dei rimbalzi è un esempio in cui il Run tab prevale sui campi legacy dei profili.

## Convenzioni di unità e coordinate

Le conversioni sono distribuite tra gli adattatori; conservare le unità ai confini esistenti e testare ogni conversione nuova.

| Grandezza | Interfaccia o modello | Unità confermata nel sorgente |
|---|---|---|
| Energia cinetica della particella ottica | Campi GUI / RunSettings | MeV; convertita in joule quando viene costruito Particle. |
| Massa della particella | RunSettings | MeV/c²; convertita in kg nel costruttore Particle. |
| Coordinate di ingresso, geometria, offset e aree attive | GUI | mm; molti adattatori dividono per 1000 per ottenere metri. |
| Angoli della GUI | GUI | gradi; convertiti in radianti per Particle e OpticalConfig. |
| Direzione, intersezioni e posizioni nel simulatore ottico | quartz_cherenkov_sim | SI: geometria/posizioni in metri, lunghezze d'onda in metri, tempi in secondi, energia in joule e massa in kg. |
| Direzione, intersezioni e posizioni interne a gamma_transport | gamma_transport | Vettori globali e geometrici in metri. |
| Energia Gamma e depositi | gamma_transport | keV; il riepilogo fornisce anche MeV per l'energia depositata per volume. |
| Coefficiente di attenuazione massico | Tabelle Gamma | cm²/g. Densità in g/cm³; il codice moltiplica per 100 per ottenere il coefficiente lineare in $m^{-1}$. |
| Range carico | electron_range_g_cm2 | g/cm². |
| Lunghezza dei segmenti di carica | ChargedTrackSegment e GammaTrack | mm; le posizioni del segmento restano in metri. |
| Geometria salvata per il viewer Gamma | gamma_run_geometry_snapshot | global_mm; gamma_event_view_data converte posizioni e segmenti da metri a mm. |
| Lunghezze d'onda nel run Cherenkov | OpticalConfig/PhotonRecord | metri nel modello; GUI in nm. Default 300–650 nm. |
| Lunghezze d'onda nel run Scintillator | ScintillatorConfig/PhotonRecord | metri nel modello; GUI in nm. Default 350–550 nm. |
| Lunghezze d'onda della luce Gamma | LightProduction | intervallo in nm. |
| Tempo di arrivo ottico | Results GUI | ns; la GUI definisce l'arrivo come ingresso fisico nella finestra finita del sensore, prima dei tagli angolari, di raccolta e QE. |

Nel run ottico la GUI documenta la convenzione: inclinazione zero significa direzione globale +Z; un'inclinazione positiva porta la traccia verso +X nel piano X-Z. _particle_for_settings forza azimuth_rad=0 anche se il campo azimuth è ancora presente nel dataclass per compatibilità. Non assumere che il campo azimuth modifichi la traiettoria GUI corrente. Invece, chiamanti diretti di Particle possono impostare anche l'azimut.

Nel trasporto Gamma i solidi sono interrogati in un frame globale in metri. RigidTransformedSolid applica la trasformazione inversa a punti e direzioni per interrogare il solido base nel suo frame locale e trasforma la normale al ritorno. Il geometry snapshot per il viewer è invece global_mm.

simulate_event colloca la sorgente Gamma all'asse X=0, Y=0, poco prima del minimo Z globale dei solidi, poi usa la direzione dal profilo. È coerente con il profilo documentato come fascio collimato centrato lungo +Z. Il punto iniziale non viene derivato da una faccia di ingresso generica: una nuova direzione obliqua o −Z richiede di rivedere e testare esplicitamente il posizionamento della sorgente.

# Algoritmi effettivamente implementati in Gamma

## 1. Probabilità di interazione

mass_attenuation_coefficients restituisce coefficienti efficaci per fotoelettrico, Compton e produzione di coppie. Le tabelle sono valori compatti tabulati per fused silica e plastic, interpolati in scala log-log. Sotto il minimo tabellato l'energia viene portata a 12.5 keV; oltre il massimo, l'interpolazione NumPy conserva il valore terminale. La produzione di coppie viene forzata a zero sotto 2 m_e c².

La normalizzazione dei materiali è significativa per le modifiche: quartz/fused silica ricadono nella tabella fused silica; plastic, PMMA, scintillator e light guide usano la tabella plastic; air restituisce coefficienti nulli. Un materiale non riconosciuto diventa custom: il suo coefficiente segue la tabella plastic e la densità predefinita è 1.0 g/cm³. Questo comportamento è una scelta del codice, non una proprietà generica dei materiali.

Per la propagazione il modulo moltiplica i coefficienti massici per la densità e per 100, ottenendo $m^{-1}$, estrae la distanza libera con una distribuzione esponenziale e confronta la distanza con l'intervallo interno al solido. Se il campione supera il confine, registra un tratto trasmesso e richiede il volume successivo lungo il raggio; se cade all'interno, seleziona il processo con probabilità proporzionale al coefficiente lineare.

## 2. Ramo di interazione e secondari

- Compton: campiona coseno di scattering tramite rejection sampling della densità Klein–Nishina, calcola l'energia Gamma diffusa con la relazione Compton e assegna la differenza di energia all'elettrone di rinculo. Il Gamma diffuso viene messo in coda se supera 1 keV; altrimenti la sua energia è conteggiata come terminata sotto soglia.
- Fotoelettrico: crea un elettrone con l'intera energia Gamma incidente. Il record del sito fotoelettrico ha deposito zero, mentre il deposito energetico è registrato lungo la traccia carica. Nel modello non sono generati cascata di fluorescenza, elettroni Auger o rilassamento atomico.
- Produzione di coppie: è permessa solo oltre 2 × 510.99895 keV. L'energia cinetica eccedente la soglia viene divisa casualmente tra elettrone e positrone; vengono poi accodati due Gamma di annichilazione da 510.99895 keV in direzioni opposte.

Le ID dei Gamma figli puntano al Gamma che li ha prodotti; le tracce cariche riportano il track ID del Gamma genitore. Questo contratto è coperto da un test mirato. max_interactions limita le interazioni Gamma; le perdite distribuite di una traccia carica non incrementano questo contatore. Se il limite viene raggiunto, l'energia dei Gamma ancora accodati viene sommata all'energia Gamma uscita: il riepilogo non ha un campo separato per distinguere questo arresto dal normale escape geometrico.

## 3. Traccia carica attraverso materiali

_charged_track mantiene una direzione rettilinea e un budget iniziale di range areale. A ogni tratto cerca il prossimo volume configurato, attraversa gli spazi senza materiale senza consumare range e registra un ChargedTrackSegment per ogni porzione solida attraversata. La perdita di energia è uniforme per unità di g/cm²: in termini del codice, è proporzionale alla frazione del budget areale usata. Il cambio di densità cambia la lunghezza fisica percorsa, non ripristina il range.

Ogni segmento registra punti iniziale/finale in metri, volume, assembly, componente, materiale, path_mm, energia cinetica prima/dopo e deposito. Il deposito è associato al volume effettivo e viene marcato interaction_site=False. Il record del processo alla posizione d'interazione ha invece interaction_site=True e deposito energetico zero. Il riepilogo conta i processi usando soltanto i record di sito, non i segmenti di perdita.

Se il range si esaurisce, l'energia residua diventa zero. Se invece il raggio non incontra altri solidi configurati, la cinetica residua è registrata come escaped_energy_keV. Un breve avanzamento numerico dopo un confine evita di scegliere ripetutamente la stessa faccia. Il selettore del prossimo volume sceglie il candidato con ingresso positivo minimo; nel codice fornito non c'è una politica di proprietà per volumi sovrapposti, quindi l'assenza di sovrapposizioni involontarie va verificata nella geometria e nei test di confine.

## 4. Luce locale, non trasporto ottico

Per un segmento in fused silica con beta > 1/1.47, _charged_track integra numericamente sim.frank_tamm_spectrum tra 300 e 650 nm e moltiplica l'integrale per la lunghezza del segmento. Questo produce un'aspettativa di fotoni Cherenkov; il conteggio campionato è Poisson e viene limitato da max_light_samples nel singolo record.

Per materiali mappati a plastic, l'aspettativa scintillante è energia depositata in MeV per la resa del volume, predefinita a 10.000 fotoni/MeV nel relativo adattatore. Il record usa la banda 350–550 nm. Il massimo campionato è applicato al record, mentre expected_photons rimane l'aspettativa calcolata.

LightProduction porta transport_status=produced_local_only. Non ci sono traiettorie di fotoni ottici, trasporto fibra/guide, assorbimento ottico, efficienza del sensore o tempi di arrivo in questo ramo.

### 5. Range elettronico: non cambiare come se fosse validato

electron_range_g_cm2 conserva questa relazione legacy, con E in MeV e output in g/cm²:

```
E = max(kinetic_energy_keV / 1000, 1e-6)
R = max(1e-5, 0.412 * E^1.265 - 0.0954 * ln(max(E, 0.001)))
```

La diagnosi allegata evidenzia risultati non monotoni: 12.5 keV → 0.419657799 g/cm², 100 keV → 0.242048532 g/cm², 300 keV → 0.204696081 g/cm², 1 MeV → 0.412 g/cm². Sono output della formula in uso, non valori di riferimento. Il test mantiene visibile la discrepanza e non dichiara accuratezza. Prima di cambiare la formula servono dati di range CSDA tracciabili per elettroni e positroni, nel dominio 12.5 keV–2.5 MeV e per i materiali modellati, con unità, fonte e ipotesi di composizione/densità.

## Energia, riepiloghi e file prodotti

GammaEvent.energy_accounting e GammaTransportResult.summary separano energia iniziale, depositata, uscita come carica, uscita come Gamma e Gamma terminata sotto soglia. Il residuo è energia iniziale meno i quattro contributi. L'energia depositata totale deriva dai record EnergyDeposit; nei test la chiusura è verificata contando le perdite delle tracce cariche una sola volta e aggiungendo le uscite Gamma.

Il riepilogo aggrega conteggi di interazione e depositi per volume e assembly, oltre alle aspettative di luce. I record di segmento non vengono interpretati come nuove interazioni. Anche ogni evento serializzato contiene energy_accounting.

Per la modalità Gamma la GUI salva gamma_results.json con eventi e snapshot geometrico, gamma_transport.png e summary.json; mantiene inoltre un indice dell'ultimo risultato completato. Un run interrotto non sovrascrive il risultato Gamma completo più recente nella vista. gamma_transport.simulate crea una sequenza casuale figlia per ciascun evento a partire dal seed; lo stop cooperativo è controllato tra gli eventi, non nel ciclo delle interazioni di un singolo evento.

Nel percorso ottico, il GUI salva summary.json e, quando applicabile, quartz_cherenkov.root, readout_arrivals.json, PNG per evento e grafici cumulativi/di distribuzione. ROOT dipende dalle librerie opzionali controllate da check_root_dependencies; l'analisi readout è persistita separatamente in readout-arrivals-v1. I campi specifici del summary sono definiti da SimulationResult.summary e aggregate_summary nel modulo ottico.

Il viewer Gamma converte i dati salvati da metri a millimetri. Preferisce i segmenti carichi dettagliati quando presenti e mantiene una fallback per il formato precedente che conteneva una sola traccia aggregata. gamma_geometry_view_data accetta solo lo schema gamma-global-assembly-geometry-v1 e richiede entrambe le tipologie di assembly, Cherenkov e Scintillator; senza snapshot valido il viewer non inventa solidi.

## Validazione e regressioni

I test mirati in test_gamma_transport.py coprono. Non è presente una suite unitaria separata chiamata test_quartz_cherenkov_sim.py; le regressioni ottiche/geometriche pertinenti sono raccolte nel checker headless:

1. la discrepanza nota della formula di range, senza asserzione di accuratezza rispetto a un riferimento;
2. chiusura dell'energia per fotoelettrico, Compton e produzione di coppie;
3. energia in uscita quando scappa una carica o un Gamma;

4. conteggio singolo dell’energia Gamma Compton sotto soglia;
5. genealogia delle ID tra Gamma e secondari carichi;
6. attraversamento di due volumi contigui, segmenti e depositi per materiale;
7. residuo di energia carica dopo l’uscita dagli ultimi volumi configurati;
8. separazione tra conteggio dei siti d’interazione e segmenti di perdita.

Il comando focale indicato in README_v78.md è:

```
python -m unittest -v test_gamma_transport
```

Il checker indicato dal README è:

```
python check_environment.py
```

check_environment.py compila i moduli elencati e richiama test Gamma mirati, controlli del viewer Gamma e molte regressioni headless su GUI, geometria, profili, trasformazioni, contabilità sensore, risultati e layout. Controlla inoltre file del pacchetto e launcher se presenti. Il sorgente include controlli specifici su detector mode e sampling, round-trip delle geometrie/terminali, trasformazioni rigide e allineamento della traccia globale, geometrie attive circolari/ellittiche, Mylar, guide, scene evento, contabilità del sensore, analisi degli arrivi, viewer statici/interattivi Gamma, progress/status e layout responsivo. Il README v78 segnala che non era disponibile una sessione desktop per QA visiva e che i launcher Windows originali non erano stati inseriti nel package.

La guida non ha eseguito le suite del simulatore. L’ispezione statica conferma che i moduli ottici e geometrici richiesti sono presenti, ma check_environment.py controlla anche file del rilascio non allegati: ad esempio il profilo Gamma, tabelle/esempi, README e riepiloghi storici. In main, source_checks viene chiamato prima delle regressioni e richiede tali file; archive e launcher sono invece saltati condizionalmente se assenti. Quindi i controlli qui elencati descrivono la copertura definita dal sorgente, non un esito ottenuto in questa sessione.

### Test da aggiungere quando cambia il modello

Mantenere i test deterministici e piccoli. Per una modifica alle tabelle o alla sorgente aggiungere casi su normalizzazione del materiale, estremi energetici, soglie, schema e pesi. Per geometria e trasformazioni testare intervalli di raggio, normali, attraversamento delle interfacce, direzioni oblique, shape/asse e volume scelto quando due intervalli cominciano vicini. Per il trasporto ottico testare campionamento deterministico con seed, soglia e aspettativa Frank–Tamm, Fresnel/Snell, TIR, assorbimento/scattering, terminali e apertura attiva del sensore. Per la traccia carica mantenere invarianti separati: somma segmenti = path del track, somma depositi = deposito del track, deposito + residuo escaped = energia iniziale. Per i rami d’interazione verificare i parent ID e la chiusura evento. Se cambia un campo salvato, verificare anche deserializzazione e viewer legacy.

Ogni formula fisica nuova dovrebbe avere test puntuali con fonte di riferimento dichiarata, oltre ai test di bookkeeping. Un test che congela il comportamento attuale dimostra una regressione rispetto al codice precedente; da solo non dimostra che il modello descriva la fisica corretta.

## Workflow consigliato per una modifica sicura

1. Scegliere il livello da cambiare. Campi/modalità nel GUI; emissione e trasporto Cherenkov/Scintillator in quartz_cherenkov_sim; sezioni, terminali, sensori e trasformazioni canoniche in quartz_geometry; coefficienti, interazioni e tracce Gamma in gamma_transport; editing/preview storico in quartz_geometry_editor.
2. Seguire il dato in entrambe le direzioni. Per una nuova impostazione tracciare valore GUI → validazione → RunSettings → snapshot → ricostruzione worker → chiamata del modello → riepilogo/persistenza. Per output nuovo, cercare anche serializer, tabella e viewer.
3. Preservare il confine del worker. Non passare Tk StringVar o modelli mutabili vivi al thread. Aggiornare la forma serializzata e il ripristino, non soltanto il costruttore della GUI.

4. Fissare le unità all’interfaccia. Se una GUI espone mm e il trasporto riceve m, fare la conversione nel punto d’adattamento e aggiungere un test numerico che attraversi la conversione. Non cambiare un campo da mm a m senza rinominare o documentare l’interfaccia.
5. Aggiornare i due consumatori di geometria Gamma. Il run GUI genera solidi di trasporto e geometry snapshot dal medesimo stato congelato. Una modifica geometrica deve mantenere coerenti _gamma_snapshot_volumes e gamma_run_geometry_snapshot, altrimenti il trasporto e la vista possono divergere.
6. Mantenere i formati salvati leggibili. summary.json normalizza vecchi campi diagnostici; il viewer conserva la fallback per tracce aggregate precedenti. Aggiungere campi con default compatibili e testare almeno un payload precedente.
7. Controllare bilanci e contatori in modo indipendente. Una perdita di energia lungo un elettrone non è una nuova interazione Gamma. Non sommare sia i depositi segmentati sia il deposito aggregato del track per chiudere l’evento.
8. Eseguire prima la suite focalizzata, poi il checker completo nel checkout completo del progetto. Per una modifica UI/geometria, aggiungere una prova manuale con GUI per comportamento visivo e interazione: il checker è headless.
9. Documentare il limite residuo. Se manca un input di riferimento o un file del package, registrarlo come blocco esplicito. Non trasformare una regressione numerica interna in una dichiarazione di accuratezza fisica.

## Limiti che devono rimanere visibili

- Lo stato Gamma è first_stage_effective_cross_sections_no_geant4: i coefficienti sono tabelle efficaci compatte, non un sostituto Geant4 o una validazione sperimentale.
- La traccia carica è rettilinea; non include multiple scattering né integrazione di stopping power dipendente dall’energia. Il range legacy è un unico budget areale con perdita uniforme per g/cm².
- Non ci sono rilassamento atomico dettagliato, cascata di fluorescenza/Auger, trasporto ottico, trasporto in fibra/guide, quenching o risposta elettronica.
- La stessa formula di range viene usata per elettroni e positroni e non è stata validata come comune ai due.
- La sorgente Gamma GUI si ancora a X=Y=0 e prima del minimo Z: la direzione obliqua/non +Z non ridefinisce automaticamente una faccia iniziale. Il profilo JSON effettivo non è presente tra gli allegati.
- La logica mostrata per il prossimo volume non contiene una regola esplicita di composizione/precedenza per solidi sovrapposti.
- Nel modello ottico il Fresnel è non polarizzato e lo scattering Rayleigh è isotropo; non c’è una variabile di polarizzazione dei fotoni. Gli indici ottici dipendono dal materiale secondo le regole effettive di SolidGeometry, che non costituiscono una tabella di dispersione calibrata per ogni nome materiale. La griglia di emissione e l’integrazione sono numeriche. I sorgenti non dichiarano una validazione sperimentale del risultato ottico.
- La geometria composita ottica rappresenta catene a sezioni lungo X e impone interfacce geometriche compatibili nel modello ordinario; non è un kernel CSG generale. I casi non rappresentabili vanno rifiutati o modellati esplicitamente, non approssimati silenziosamente.
- La GUI ha modalità separate: il trasporto Gamma genera solo stime di luce locale, mentre l’ottica Cherenkov/Scintillator è un altro run e un altro formato di record.
- Il checker e i test non provano la correttezza fisica delle tabelle Gamma né la validità sperimentale del trasporto ottico. Il test di range preserva intenzionalmente un comportamento anomalo; il QA visivo desktop non era disponibile nel rilascio documentato. Il checker completo richiede inoltre artefatti di rilascio assenti dalla cartella qui analizzata.

# Glossario

| Termine | Significato in questa guida |
|---|---|
| Assembly / assemblaggio | Gruppo geometrico Cherenkov o Scintillator, con solidi attivi, eventuali guide, sensori e trasformazione globale. |
| Areal range / range areale | Spessore integrato in g/cm²; combinato con la densità determina una distanza in un dato materiale. |
| AssemblyGeometryModel | Modello canonico GUI in mm per sezioni attive, guida, terminali e scene del fotosensore. |
| CompoundGeometry | Sequenza di SolidGeometry contigui lungo X con interfacce interne esplicite. |
| Detector crossing / readout arrival | Uscita attraverso la finestra attiva; può precedere i tagli di accettanza, raccolta e QE. |
| Fixed / Poisson sampling | fixed fissa il numero del campione Monte Carlo; Poisson estrae il numero con la resa attesa come media. |
| GeometryModel | Modello storico dell'editor a sezioni; non è l'oggetto canonico del trasporto GUI. |
| OpticalConfig | Dataclass del simulatore per campionamento, interazioni ottiche e risposta dei sensori. |
| SolidGeometry | Solido convesso parametrico del trasporto ottico, con unità SI. |
| ChargedTrackSegment | Parte della traccia di elettrone/positrone dentro uno specifico volume materiale. |
| CSDA | Continuous Slowing Down Approximation; qui è il nome della relazione di range approssimata, non un trasporto dettagliato. |
| beta | Rapporto tra velocità della particella e velocità della luce, usato per la soglia Cherenkov locale. |
| Deposito d'interazione | Record del sito fotoelettrico, Compton o coppia, distinto dai depositi distribuiti dei segmenti carichi. |
| Deposito di segmento | Energia sottratta alla carica in un tratto di volume; marcato interaction_site=False. |
| Gamma figlio | Gamma diffuso o di annichilazione accodato da un'interazione; conserva parent_track_id. |
| Interazione | Evento discreto Gamma contato in interaction_counts; non è un segmento di perdita carica. |
| RunSnapshot | Copia serializzata e immutabile di impostazioni, modelli e geometria passata al worker. |
| Track / traccia | Percorso carico o segmento Gamma registrato nei risultati; le posizioni interne sono in metri. |
| Trasporto locale della luce | Calcolo di aspettativa e conteggio campionato della luce sul tratto carico, senza simulare il viaggio dei fotoni ottici. |

# Fonti usate

Questa guida è basata sui file allegati: quartz_cherenkov_gui.py, quartz_cherenkov_sim.py, quartz_geometry.py, quartz_geometry_editor.py, gamma_transport.py, README_v78.md, README_gamma_transport_stage.md, implementation_summary_v78.txt, gamma_range_model_diagnostic.md, test_gamma_transport.py e check_environment.py. Non sono riportati numeri di riga, perché non sono necessari per identificare le fonti e non vanno ricostruiti da copie diverse.